شرح تفصيلي لكود مساعد Gemini AI

هذا المستند يقدم شرحًا مفصلاً لكل سطر، دالة، وخاصية في الكود البرمجي المرفق، مع التركيز على وظيفته وأهميته.

1. الاستيرادات (Imports)

Python

```
#ctrl , == settings
import enum
import nltk
```

- **السطر 1:** `#ctrl , == settings`
- هذا السطر هو تعليق (comment) ولا يؤثر على تنفيذ الكود. يبدو أنه تذكير للمطور حول اختصار لوحة المفاتيح لفتح الإعدادات في بيئة تطوير معينة.
- **السطر 2:** `import enum`
- يستورد هذا السطر الوحدة `enum` في بايثون. تُستخدم هذه الوحدة لإنشاء تعدادات (enumerations)، وهي مجموعة من الأسماء الرمزية (أعضاء) المرتبطة بقيم فريدة وثابتة. على الرغم من استيرادها، لا يبدو أنها مستخدمة بشكل مباشر في الجزء المرفق من الكود.
- **السطر 3:** `import nltk`
- يستورد هذا السطر مكتبة (Natural Language Toolkit) `nltk`، وهي مكتبة قوية للعمل مع البيانات اللغوية في بايثون. تُستخدم هذه المكتبة بشكل مكثف في هذا الكود لمعالجة النصوص، مثل تقسيم الجمل إلى كلمات (tokenization) وتصغير الكلمات (stemming).

2. تحميل بيانات NLTK

Python

```
try:
    nltk.data.find(\'tokenizers/punkt\')
    nltk.data.find(\'tokenizers/punkt_tab\')
except LookupError:
    nltk.download(\'punkt\')
    nltk.download(\'punkt_tab\')
```

- **السطر 5:** `try:`
- يبدأ هذا السطر كتلة `try-except`، وهي آلية للتعامل مع الأخطاء. يحاول الكود داخل كتلة `try` التنفيذ، وإذا حدث خطأ معين (محدد في `except`)، يتم تنفيذ الكود داخل كتلة `except`.

- **السطر 6**: `nltk.data.find('tokenizers/punkt')`
- يحاول هذا السطر البحث عن بيانات `punkt` الخاصة بـ `punkt`. NLTK هو مُقسم جمل (sentence tokenizer) يستخدم لتقسيم النص إلى جمل. إذا لم يتم العثور على البيانات، فسيتم طرح خطأ `LookupError`.
- **السطر 7**: `nltk.data.find('tokenizers/punkt_tab')`
- على غرار السطر السابق، يحاول هذا السطر البحث عن بيانات `punkt_tab`. يبدو أنه نوع آخر من مُقسم الجمل أو بيانات إضافية مرتبطة بـ `punkt`.
- **السطر 8**: `except LookupError:`
- إذا تم طرح خطأ `LookupError` في أي من السطرين 6 أو 7 (مما يعني أن بيانات NLTK المطلوبة غير موجودة)، فسيتم تنفيذ الكود داخل هذه الكتلة.
- **السطر 9**: `nltk.download('punkt')`
- يقوم هذا السطر بتنزيل بيانات `punkt` إذا لم يتم العثور عليها مسبقًا. هذا يضمن أن المُقسم متاح للاستخدام.
- **السطر 10**: `nltk.download('punkt_tab')`
- يقوم هذا السطر بتنزيل بيانات `punkt_tab` إذا لم يتم العثور عليها مسبقًا.

3. إعداد أداة تصغير الكلمات (Stemmer)

Python

```
13  from nltk.stem.lancaster import LancasterStemmer
14  stemmer = LancasterStemmer()
```

- **السطر 13**: `from nltk.stem.lancaster import LancasterStemmer`
- يستورد هذا السطر فئة `LancasterStemmer` من وحدة `nltk.stem.lancaster`. `LancasterStemmer` هي خوارزمية لتصغير الكلمات (stemming)، والتي تقلل الكلمات إلى جذرها أو شكلها الأساسي (على سبيل المثال، `running` تصبح `run`).
- **السطر 14**: `stemmer = LancasterStemmer()`
- ينشئ هذا السطر كائنًا (instance) من `LancasterStemmer`. سيتم استخدام هذا الكائن لاحقًا لتصغير الكلمات في النص.

4. استيرادات إضافية (Additional Imports)

Python

```
16  import numpy as np
17  import tensorflow as tf
18  import random
19  import json
20  import pickle
21  from time import sleep
22  import streamlit as st
```

- **السطر 16:** `import numpy as np`
- يستورد هذا السطر مكتبة `numpy` ، وهي مكتبة أساسية للحوسبة العلمية في بايثون، وتوفر دعمًا للمصفوفات الكبيرة متعددة الأبعاد والكائنات المصفوفة، بالإضافة إلى مجموعة من الدوال عالية المستوى للعمل على هذه المصفوفات. يتم استيرادها بالاسم المستعار `np` .
- **السطر 17:** `import tensorflow as tf`
- يستورد هذا السطر مكتبة `tensorflow` ، وهي إطار عمل مفتوح المصدر للتعلم الآلي والشبكات العصبية. تُستخدم هنا لبناء وتدريب نموذج الشبكة العصبية للمساعد الذكي.
- **السطر 18:** `import random`
- يستورد هذا السطر الوحدة `random` ، التي توفر دوال لتوليد أرقام عشوائية. تُستخدم هنا لاختيار رد عشوائي من قائمة الردود المتاحة للمساعد الذكي.
- **السطر 19:** `import json`
- يستورد هذا السطر الوحدة `json` ، التي تُستخدم للعمل مع بيانات JSON (JavaScript Object Notation). تُستخدم هنا لتحميل ملف `intent.json` الذي يحتوي على أنماط ونوايا المحادثة.
- **السطر 20:** `import pickle`
- يستورد هذا السطر الوحدة `pickle` ، التي تُستخدم لتسلسل (serialize) وإلغاء تسلسل (deserialize) كائنات بايثون. تُستخدم هنا لحفظ وتحميل البيانات المعالجة (الكلمات، التصنيفات، بيانات التدريب) لتجنب إعادة معالجتها في كل مرة يتم فيها تشغيل البرنامج.
- **السطر 21:** `from time import sleep`
- يستورد هذا السطر الدالة `sleep` من الوحدة `time` . تُستخدم `sleep` لإيقاف تنفيذ البرنامج مؤقتًا لعدد محدد من الثواني. تُستخدم هنا لإضافة تأخير بسيط قبل عرض رد المساعد الذكي، مما يحاكي تجربة محادثة أكثر طبيعية.
- **السطر 22:** `import streamlit as st`
- يستورد هذا السطر مكتبة `streamlit` ، وهي إطار عمل مفتوح المصدر لإنشاء تطبيقات ويب تفاعلية للتعلم الآلي وعلوم البيانات بسرعة. تُستخدم هنا لبناء واجهة المستخدم الرسومية (GUI) للمساعد الذكي.

5. تحميل بيانات النوايا (Intent Data Loading)

Python

```
24  with open("intent.json") as file:
25      data = json.load(file)
```

- **السطر 24:** `with open("intent.json") as file:`
- يفتح هذا السطر ملفًا باسم `intent.json` في وضع القراءة (`"r"` هو الوضع الافتراضي إذا لم يتم تحديده). يضمن استخدام `with` أن يتم إغلاق الملف تلقائيًا بعد الانتهاء من العمل عليه، حتى لو حدث خطأ.
- **السطر 25:** `data = json.load(file)`
- يقرأ هذا السطر محتويات ملف `intent.json` (الذي يجب أن يكون بتنسيق JSON) ويقوم بتحميله في متغير `data` ككائن بايثون (عادةً قاموس أو قائمة).

6. إعداد بيانات التدريب (Training Data Preparation)

Python

```
27  try:
28      with open("data.pickle", "rb") as f:
29          words, labels, training, output = pickle.load(f)
30  except:
31      words = []
32      label = []
33      docs_x = []
34      docs_y = []
35
36      for intent in data["intents"]:
37         for patterns in intent["patterns"]:
38             wrds = nltk.word_tokenize(patterns)
39             words.extend(wrds)
40             docs_x.append(wrds)
41             docs_y.append(intent["tag"])
42
43         if intent["tag"] not in label:
44             label.append(intent["tag"])
45
46      words = [stemmer.stem(w.lower()) for w in words if w != "?"]
47      words = sorted(list(set(words)))
48
49      labels = label
50      training = []
51      output = []
52
53      out_empty = [0 for _ in range(len(labels))]
54
```

```
    for x, doc in enumerate(docs_x):
        bag = []
        wrds = [stemmer.stem(w) for w in doc]

        for w in words:
            if w in wrds:
                bag.append(1)
            else:
                bag.append(0)

        output_row = list(out_empty)
        output_row[labels.index(docs_y[x])] = 1

        training.append(bag)
        output.append(output_row)

    training = np.array(training)
    output = np.array(output)

    with open("data.pickle", "wb") as f:
        pickle.dump((words, labels, training, output), f)
```

- **السطر 27-29:** `try...pickle.load(f)`
- يحاول هذا الجزء من الكود تحميل البيانات المعالجة مسبقًا (`words` , `labels` , `training` , `output`) من ملف `data.pickle` . إذا كان الملف موجودًا وتم تحميله بنجاح، فسيتم تخطي خطوة معالجة البيانات.
- **السطر 30:** `except:`
- إذا فشل تحميل ملف `data.pickle` (على سبيل المثال، إذا لم يكن الملف موجودًا أو كان تالفًا)، فسيتم تنفيذ الكود داخل كتلة `except` ، مما يعني أن البيانات سيتم معالجتها من البداية.
- **السطر 31-34:** `docs_y` , `docs_x = []` , `label = []` , `words = []` =
- يتم تهيئة قوائم فارغة لتخزين الكلمات الفريدة، التصنيفات (tags)، أنماط المستندات (docs_x)، وتصنيفات المستندات المقابلة (docs_y).
- **السطر 36:** `for intent in data["intents"]:`
- يبدأ هذا التكرار بالمرور عبر كل نية (intent) في قائمة `intents` الموجودة في ملف `data` المحمل من `intent.json` .
- **السطر 37:** `for patterns in intent["patterns"]:`
- داخل كل نية، يتم المرور عبر كل نمط (pattern) من أنماط الجمل المرتبطة بهذه النية.
- **السطر 38:** `wrds = nltk.word_tokenize(patterns)`
- يستخدم `nltk.word_tokenize` لتقسيم كل نمط إلى قائمة من الكلمات (tokens).

- **السطر 39:** `words.extend(wrds)`
- يتم إضافة الكلمات المستخرجة من النمط الحالي إلى قائمة `words` الكلية.
- **السطر 40:** `docs_x.append(wrds)`
- يتم إضافة قائمة الكلمات (النمط) إلى `docs_x` ، والتي ستمثل المدخلات لتدريب النموذج.
- **السطر 41:** `docs_y.append(intent["tag"])`
- يتم إضافة `tag` (التصنيف) الخاص بالنية الحالية إلى `docs_y` ، والذي سيمثل المخرجات المستهدفة لتدريب النموذج.
- **السطر 43-44:** `if intent["tag"] not in label: label.append(intent["tag"])`
- يضيف هذا الكود التصنيف (tag) إلى قائمة `label` إذا لم يكن موجودًا بالفعل، مما يضمن أن `label` تحتوي على جميع التصنيفات الفريدة.
- **السطر 46:** `words = [stemmer.stem(w.lower()) for w in words if w != "?"]`
- يتم هنا معالجة قائمة `words` : يتم تحويل كل كلمة إلى أحرف صغيرة (`w.lower()`)، ثم يتم تصغيرها باستخدام `stemmer.stem()` ، ويتم استبعاد علامات الاستفهام (`?`). الهدف هو توحيد الكلمات وتقليلها إلى جذورها.
- **السطر 47:** `words = sorted(list(set(words)))`
- يتم تحويل قائمة `words` إلى مجموعة (`set`) لإزالة الكلمات المكررة، ثم يتم تحويلها مرة أخرى إلى قائمة ويتم فرزها أبجديًا. هذا ينتج قائمة فريدة ومرتبة من جميع الكلمات الأساسية الموجودة في بيانات التدريب.
- **السطر 49:** `labels = label`
- يتم تعيين قائمة `label` (التي تحتوي على التصنيفات الفريدة) إلى المتغير `labels` .
- **السطر 50-51:** `output` , `training = []` = `[]`
- يتم تهيئة قوائم فارغة لـ `training` (مدخلات التدريب) و `output` (مخرجات التدريب).
- **السطر 53:** `out_empty = [0 for _ in range(len(labels))]`
- يتم إنشاء قائمة `out_empty` ، وهي قائمة من الأصفار بحجم عدد التصنيفات. ستُستخدم هذه القائمة لإنشاء تمثيل

one-hot encoding للمخرجات.

- **السطر 55:** `for x, doc in enumerate(docs_x):`
- يبدأ هذا التكرار بالمرور عبر كل مستند (قائمة كلمات) في `docs_x` ، مع الحصول على الفهرس `x` والمستند `doc` .
- **السطر 56:** `bag = []`
- يتم تهيئة قائمة فارغة تسمى `bag` لكل مستند. ستمثل هذه القائمة "كيس الكلمات" (bag of words) للمستند الحالي.
- **السطر 57:** `wrds = [stemmer.stem(w) for w in doc]`

- يتم تصغير كل كلمة في المستند الحالي `doc` باستخدام `stemmer` .
- **السطر 59-63:** `for w in words: if w in wrds: bag.append(1) else: bag.append(0)`
- يتم إنشاء تمثيل "كيس الكلمات" للمستند. لكل كلمة فريدة في قائمة `words` الكلية، إذا كانت الكلمة موجودة في المستند المصغر `wrds` ، يتم إضافة `1` إلى `bag` ؛ وإلا، يتم إضافة `0` .
- **السطر 65:** `output_row = list(out_empty)`
- يتم إنشاء نسخة من `out_empty` (قائمة الأصفار) لتمثيل صف الإخراج الحالي.
- **السطر 66:** `output_row[labels.index(docs_y[x])] = 1`
- يتم تعيين `1` في `output_row` عند الفهرس الذي يتوافق مع تصنيف المستند الحالي (`docs_y[x]`). هذا يحول التصنيف إلى تمثيل one-hot encoding.
- **السطر 68:** `training.append(bag)`
- يتم إضافة "كيس الكلمات" للمستند الحالي إلى قائمة `training` .
- **السطر 69:** `output.append(output_row)`
- يتم إضافة صف الإخراج (one-hot encoded) للمستند الحالي إلى قائمة `output` .
- **السطر 71:** `training = np.array(training)`
- يتم تحويل قائمة `training` إلى مصفوفة NumPy. هذا ضروري لتدريب نموذج TensorFlow.
- **السطر 72:** `output = np.array(output)`
- يتم تحويل قائمة `output` إلى مصفوفة NumPy.
- **السطر 74-75:** `with open("data.pickle", "wb") as f: pickle.dump((words, labels, training, output), f)`
- يتم حفظ البيانات المعالجة (`words` , `labels` , `training` , `output`) في ملف `data.pickle` باستخدام `pickle.dump()` . يتم فتح الملف في وضع الكتابة الثنائية (`"wb"`). هذا يسمح بتحميل البيانات بسرعة في المستقبل دون الحاجة إلى إعادة معالجتها.

7. بناء وتدريب النموذج (Model Building and Training)

Python

```
77  def build_and_train_model():
78      model = tf.keras.Sequential([
79          tf.keras.layers.Dense(8, input_shape=(len(training[0]),),activation
80          tf.keras.layers.Dense(8, activation=\'relu\'),
81          tf.keras.layers.Dense(len(output[0]), activation=\'softmax\')
82      ])
83
84      model.compile(optimizer=\'adam\', loss=\'categorical_crossentropy\',met
85      model.fit(training, output, epochs =1000, batch_size=8, verbose=1)
```

```
86      model.save("chatbot_model.h5")
87      return model
```

- **السطر 77:** `def build_and_train_model():`
- تعريف دالة تسمى `build_and_train_model` ، وهي مسؤولة عن بناء وتدريب نموذج الشبكة العصبية.
- **السطر 78:** `model = tf.keras.Sequential([ ... ])`
- ينشئ هذا السطر نموذج شبكة عصبية تسلسلية (Sequential model) باستخدام Keras (جزء من TensorFlow). النموذج التسلسلي هو مكدس خطي من الطبقات.
- **السطر 79:** `tf.keras.layers.Dense(8, input_shape=(len(training[0]),),activation=\'relu\')`
- هذه هي الطبقة المخفية الأولى (hidden layer) في الشبكة. إنها طبقة `Dense` (متصلة بالكامل) تحتوي على 8 وحدات عصبية (neurons). `input_shape` يحدد شكل المدخلات لهذه الطبقة، وهو طول متجه "كيس الكلمات" (`len(training[0])`). دالة التنشيط المستخدمة هي `relu` (Rectified Linear Unit).
- **السطر 80:** `tf.keras.layers.Dense(8, activation=\'relu\')`
- هذه هي الطبقة المخفية الثانية، وهي أيضًا طبقة `Dense` تحتوي على 8 وحدات عصبية وتستخدم دالة التنشيط `relu` .
- **السطر 81:** `tf.keras.layers.Dense(len(output[0]), activation=\'softmax\')`
- هذه هي طبقة الإخراج (output layer). تحتوي على عدد من الوحدات العصبية يساوي عدد التصنيفات الفريدة (`len(output[0])`). دالة التنشيط المستخدمة هي `softmax` ، والتي تحول المخرجات إلى توزيع احتمالي، مما يجعلها مناسبة لتصنيف الفئات المتعددة.
- **السطر 84:** `model.compile(optimizer=\'adam\', loss=\'categorical_crossentropy\',metrics=[\'accuracy\'])`
- يقوم هذا السطر بتجميع النموذج. يتم تحديد المحسن (optimizer) بـ `adam` ، وهو خوارزمية تحسين شائعة. دالة الخسارة (loss function) هي `categorical_crossentropy` ، وهي مناسبة لتصنيف الفئات المتعددة مع مخرجات one-hot encoded. يتم تتبع `accuracy` كمقياس أثناء التدريب.
- **السطر 85:** `model.fit(training, output, epochs =1000, batch_size=8, verbose=1)`
- يقوم هذا السطر بتدريب النموذج باستخدام بيانات `training` كمدخلات و `output` كمخرجات مستهدفة. يتم التدريب لمدة 1000 `epochs` (مرور كامل عبر مجموعة البيانات)، وحجم الدفعة (batch size) هو 8. `verbose=1` يعني أن التقدم سيتم عرضه أثناء التدريب.
- **السطر 86:** `model.save("chatbot_model.h5")`
- بعد التدريب، يتم حفظ النموذج المدرب في ملف يسمى `chatbot_model.h5` . هذا يسمح بإعادة استخدام النموذج دون الحاجة إلى إعادة تدريبه في كل مرة.
- **السطر 87:** `return model`
- تعيد الدالة النموذج المدرب.

8. تحميل أو بناء النموذج (Model Loading or Building)

```python
try:
    model = tf.keras.models.load_model("chatbot_model.h5")
except:
    model = build_and_train_model()
```

- **السطر 89-90:** `try...tf.keras.models.load_model("chatbot_model.h5")`
- يحاول هذا الجزء من الكود تحميل نموذج الشبكة العصبية المدرب مسبقًا من ملف `chatbot_model.h5`. إذا كان النموذج موجودًا وتم تحميله بنجاح، فسيتم استخدامه مباشرة.
- **السطر 91:** `except:`
- إذا فشل تحميل النموذج (على سبيل المثال، إذا لم يكن الملف موجودًا)، فسيتم تنفيذ الكود داخل كتلة `except`.
- **السطر 92:** `model = build_and_train_model()`
- في حالة عدم وجود نموذج مدرب مسبقًا، يتم استدعاء الدالة `build_and_train_model()` لبناء وتدريب نموذج جديد.

9. دالة "كيس الكلمات" (Bag of Words Function)

```python
def bag_of_words(s, words):
    bag = [0 for _ in range(len(words))]
    se_words = nltk.word_tokenize(s)
    se_words = [ stemmer.stem(w.lower()) for w in se_words]

    for se in se_words:
        for i,w in enumerate(words):
            if w == se:
                bag[i] = 1
    return np.array(bag)
```

- **السطر 94:** `def bag_of_words(s, words):`
- تعريف دالة تسمى `bag_of_words` تأخذ سلسلتين نصيتين: `s` (الجملة المدخلة) و `words` (قائمة الكلمات الفريدة من بيانات التدريب).
- **السطر 95:** `bag = [0 for _ in range(len(words))]`
- يتم تهيئة قائمة `bag` بأصفار، بحجم يساوي عدد الكلمات الفريدة في `words`. هذه هي المصفوفة التي ستمثل "كيس الكلمات" للجملة المدخلة.

- **السطر 96:** `se_words = nltk.word_tokenize(s)`
- يتم تقسيم الجملة المدخلة `s` إلى كلمات باستخدام `nltk.word_tokenize` .
- **السطر 97:** `se_words = [ stemmer.stem(w.lower()) for w in se_words]`
- يتم تحويل كل كلمة في `se_words` إلى أحرف صغيرة ثم تصغيرها باستخدام `stemmer` .
- **السطر 99-102:** `for se in se_words: for i,w in enumerate(words): if w == se: bag[i] = 1`
- يتم المرور عبر الكلمات المصغرة من الجملة المدخلة (`se_words`). لكل كلمة، يتم البحث عنها في قائمة `words` الكلية. إذا تم العثور عليها، يتم تعيين `1` في الموضع المقابل في قائمة `bag` .
- **السطر 103:** `return np.array(bag)`
- تعيد الدالة مصفوفة `bag` NumPy، والتي تمثل تمثيل "كيس الكلمات" للجملة المدخلة.

10. إعداد واجهة المستخدم باستخدام Streamlit (Streamlit UI Setup)

Python

```python
# ✨ GEMINI STYLE & FIXED THEME CONFIGURATION ✨
# ============================================================

st.set_page_config(page_title="Gemini AI Assistant", page_icon="✨" , layou

with st.sidebar:
    st.markdown(\'### 🪐 Appearance\')
    theme_choice = st.radio("Choose Mode:", ["Dark Mode", "Light Mode"])

if theme_choice == "Dark Mode":
    bg_color = "#111111"
    text_color ="#FFFFFFFF"
    chat_text_color ="#FFFFFFFF"
    input_bg = "#111111"

else:
    bg_color = "#FFFFFF"
    text_color = "#111111"
    chat_text_color = "#111111"
    input_bg = "#FFFFFF"

st.markdown(f"""
    <style>
    .stApp {{
        background-color: {bg_color} !important;
        color: {text_color} !important;
    }}
```

```
    .gemini-title {{
         background: linear-gradient(70deg, #4285f4, #9b51e0, #e24a8a);
                background-size: 200% 200%;
                animation: geminiGlow 6s ease infinite;
                -webkit-background-clip: text;
                -webkit-text-fill-color: transparent;
                font-family: \'Google Sans\', sans-serif;
                font-weight: 500;
                font-size: 2.8rem;
                text-align: left;
                margin-bottom: 20px;
            }}
            .stChatMessage {{
                animation: fadeIn 0.4s ease-in-out;
                padding: 10px 0px !important;
            }}
            /* تعديل جذري لضمان وضوح نصوص الشات بالكامل ومقاومة التداخل اللوني المظلم */
            .stChatMessage p, .stChatMessage span, .stChatMessage div {{
                color: {chat_text_color} !important;
            }}
            @keyframes fadeIn {{
                from {{ opacity: 0; transform: translateY(5px); }}
                to {{ opacity: 1; transform: translateY(0); }}
            }}
            .stChatInputContainer textarea {{
                background-color: {input_bg} !important;
                color: {text_color} !important;
                border-radius: 28px !important;
                border: 1px solid rgba(128,128,128,0.2) !important;
            }}
            </style>
        """, unsafe_allow_html=True)

st.markdown(\'<h1 class="gemini-title">✨ Gemini AI Assistant</h1>\', unsaf
st.write("How can I help you discover or build today?")

if  "message" not in st.session_state:
    st.session_state.message = []

for message  in  st.session_state.message:
    avatar = "🤖" if message["role"] == "user" else "💬"
    with st.chat_message(message["role"],avatar=avatar):
        st.markdown(message["content"])

if  user_input := st.chat_input("Ask Gemini..."):
    with st.chat_message("user", avatar="🤖"):
        st.markdown(user_input)
    st.session_state.message.append({"role": "user", "content": user_input}
```

```
181
182     current_bag = bag_of_words(user_input, words)
183     results = model.predict(np.array([current_bag]))[0]
184     result_index = np.argmax(results)
185     tag = labels[result_index]
186
187     bot_reply = "I am sorry,I did not  understand that."
188     if  results[result_index] > 0.8:
189         for tg in data["intents"]:
190             if tg[\'tag\'] == tag:
191                 bot_reply = random.choice(tg[\'responses\'])
192     with st.chat_message("assistant", avatar="💬"):
193         with st.spinner(""):
194             sleep(0.4)
195             st.markdown(bot_reply)
196
197     st.session_state.message.append({"role": "assistant", "content": bot_re
```

- **السطر 106-107**: تعليقات توضح أن هذا القسم مخصص لتكوين واجهة المستخدم بأسلوب Gemini.
- **السطر 109**: `st.set_page_config(page_title="Gemini AI Assistant", page_icon="✨" , layout="centered")`
- يقوم هذا السطر بتكوين إعدادات صفحة Streamlit. يحدد عنوان الصفحة في المتصفح (`page_title`)، أيقونة الصفحة (`page_icon`)، وتخطيط الصفحة (`"layout="centered"`) يجعل المحتوى في المنتصف).
- **السطر 111**: `with st.sidebar:`
- ينشئ هذا السطر كتلة `with` لوضع العناصر داخل الشريط الجانبي (sidebar) لتطبيق Streamlit.
- **السطر 112**: `st.markdown(\'### 🪐 Appearance\')`
- يعرض عنوانًا فرعيًا "🪐 Appearance" في الشريط الجانبي باستخدام تنسيق Markdown.
- **السطر 113**: `theme_choice = st.radio("Choose Mode:", ["Dark Mode", "Light Mode"])`
- ينشئ زر اختيار (radio button) في الشريط الجانبي يسمح للمستخدم باختيار بين "Dark Mode" و "Light Mode". يتم تخزين الاختيار في `theme_choice`.
- **السطر 115-125**: `if theme_choice == "Dark Mode": ... else: ...`
- تتحقق هذه الكتلة من اختيار المستخدم للوضع (الداكن أو الفاتح) وتقوم بتعيين متغيرات الألوان (`input_bg`, `chat_text_color`, `text_color`, `bg_color`) وفقًا لذلك. هذه المتغيرات ستُستخدم لتطبيق الأنماط المخصصة.
- **السطر 127-164**: `st.markdown(f""" <style> ... </style> """, unsafe_allow_html=True)`
- يقوم هذا السطر بحقن كود CSS مخصص في تطبيق Streamlit لتطبيق الأنماط المرئية. يستخدم f-string لتضمين متغيرات الألوان المحددة مسبقًا. تتضمن الأنماط:
  - `stApp`: يحدد لون الخلفية ولون النص العام للتطبيق.

    - `gemini-title` : يطبق تدرجًا لونيًا وتأثير رسوم متحركة (geminiGlow) على عنوان "Gemini AI Assistant"، مما يمنحه مظهرًا مميزًا.
    - `stChatMessage` : يطبق رسومًا متحركة `fadeIn` على رسائل الدردشة ويحدد المسافات البادئة.
    - `stChatMessage p, .stChatMessage span, .stChatMessage div` : يضمن أن نصوص الدردشة تظهر بلون `chat_text_color` لضمان الوضوح.
    - `@keyframes fadeIn` : يحدد الرسوم المتحركة `fadeIn` .
    - `stChatInputContainer textarea` : يحدد لون الخلفية ولون النص ومظهر حقل إدخال الدردشة.
- `unsafe_allow_html=True` ضروري للسماح لـ Streamlit بتفسير كود HTML/CSS الخام.
- **السطر 166:** `st.markdown(\'<h1 class="gemini-title">✨ Gemini AI Assistant</h1>\', unsafe_allow_html=True)`
    - يعرض عنوانًا رئيسيًا "✨ Gemini AI Assistant" باستخدام تنسيق Markdown وكود HTML المخصص لتطبيق الأنماط المحددة في CSS.
- **السطر 167:** `st.write("How can I help you discover or build today?")`
    - يعرض رسالة ترحيب للمستخدم.
- **السطر 169-170:** `if "message" not in st.session_state: st.session_state.message = []`
    - يتحقق هذا الكود مما إذا كانت قائمة الرسائل موجودة في `st.session_state` (وهي طريقة Streamlit لتخزين الحالة عبر عمليات إعادة التشغيل). إذا لم تكن موجودة، يتم تهيئة قائمة فارغة.
- **السطر 172-175:** `for message in st.session_state.message: ... st.markdown(message["content"])`
    - يتكرر هذا الجزء عبر جميع الرسائل المخزنة في `st.session_state.message` ويعرضها في واجهة الدردشة. يتم تحديد الصورة الرمزية (`avatar`) بناءً على دور الرسالة (مستخدم أو مساعد).
- **السطر 177:** `if user_input := st.chat_input("Ask Gemini..."):`
    - ينشئ حقل إدخال للدردشة مع نص توجيهي "Ask Gemini...". إذا أدخل المستخدم نصًا، يتم تخزينه في `user_input` ويتم تنفيذ الكتلة التالية.
- **السطر 178-180:** `with st.chat_message("user", avatar="🤖"): st.markdown(user_input); st.session_state.message.append({"role": "user", "content": user_input})`
    - يعرض رسالة المستخدم في واجهة الدردشة مع الصورة الرمزية "🤖" ويضيفها إلى سجل الرسائل في `st.session_state` .
- **السطر 182:** `current_bag = bag_of_words(user_input, words)`
    - يحول إدخال المستخدم إلى تمثيل "كيس الكلمات" باستخدام الدالة `bag_of_words` التي تم تعريفها مسبقًا.
- **السطر 183:** `results = model.predict(np.array([current_bag]))[0]`
    - يستخدم النموذج المدرب (`model`) للتنبؤ بالنية بناءً على `current_bag` . يتم تحويل `current_bag` إلى مصفوفة NumPy ثنائية الأبعاد لأن `model.predict` يتوقع دفعة من المدخلات. `[0]` يأخذ النتيجة

الأولى (والوحيدة) من الدفعة.

- **السطر 184:** `result_index = np.argmax(results)`
- يجد الفهرس الذي يحتوي على أعلى قيمة احتمالية في مخرجات النموذج، وهو ما يمثل النية الأكثر احتمالاً.
- **السطر 185:** `tag = labels[result_index]`
- يستخدم الفهرس الذي تم العثور عليه لاسترداد التصنيف (tag) المقابل من قائمة `labels` .
- **السطر 187:** `bot_reply = "I am sorry,I did not understand that".`
- يتم تهيئة رد افتراضي للمساعد الذكي في حالة عدم فهم النية.
- **السطر 188-191:** `if results[result_index] > 0.8: ... bot_reply = random.choice(tg["responses"])`
- يتحقق هذا الشرط مما إذا كانت الثقة في النية المتوقعة (أعلى احتمالية) أكبر من 0.8. إذا كانت كذلك، يتم البحث عن النية المطابقة في `data["intents"]` ويتم اختيار رد عشوائي من قائمة الردود المرتبطة بهذه النية.
- **السطر 192-195:** `with st.chat_message("assistant", avatar="💬"): with st.spinner(""): sleep(0.4); st.markdown(bot_reply)`
- يعرض رد المساعد الذكي في واجهة الدردشة مع الصورة الرمزية "💬". يتم عرض مؤشر تحميل (`st.spinner`) وتأخير بسيط (`sleep(0.4)`) لمحاكاة التفكير قبل عرض الرد.
- **السطر 197:** `st.session_state.message.append({"role": "assistant", "content": bot_reply})`
- يضيف رد المساعد الذكي إلى سجل الرسائل في `st.session_state` .

المراجع

[1] NLTK (Natural Language Toolkit). (n.d.). Retrieved from

[2] TensorFlow. (n.d.). Retrieved from

[3] Streamlit. (n.d.). Retrieved from

[4] NumPy. (n.d.). Retrieved from